

1 Discrimination d'une image

L'objectif de ce mini-projet est de réaliser un système discrimination d'images réelles. La base de données d'images est une base de données classique de test (base Wang) et est composée de 10 types d'images : Jungle, Plage, Monuments, Bus, Dinosaures, Eléphants, Fleurs, Chevaux, Montagne et Plats.

Bien sûr à l'intérieur de chaque classe (100 images par classe), les images présentent une forte variabilité. Je vous propose de mettre en place une application de classification. Dans le cadre du système de classification, notre système de base de données d'images fonctionnera de la façon suivante :

- l'utilisateur proposera au système une nouvelle image "inconnue" ;
- le système affichera alors la classe d'appartenance de l'image.

Afin de mettre en place cette, nous devons mesurer des paramètres sur chaque image afin de caractériser ces différentes images. Vous allez développer deux approches, l'une reposant sur des descripteurs "Image" l'autre suivant une stratégie Deep donc à base de descripteurs CNN.

1.1 Approche basée Descripteurs (basée modèle)

1.1.1 Calcul des descripteurs

Cette étape n'est pas forcément complexe mais nécessite du temps de développement et des connaissances en traitement d'images. Dans le cadre de ce projet, nous vous fournissons directement les mesures de caractéristiques préalablement faites sur les images de la base Wang dans un fichier Excel. Nous proposons d'utiliser la mesure image FCTH.

Nous fournissons l'article fondateur expliquant le principe de ces mesures.

Nous obtenons un tableau de mesures 1000 lignes pour les 1000 images. Les mesures sont stockées dans le fichier wang.xlsx.

Attention à l'ordre de rangement des images : comme nous l'avons dit, la base est composée de 10 types d'images : Jungle, Plage, Monuments, Bus, Dinosaures, Eléphants, Fleurs, Chevaux, Montagne et Plats. Comme vous pouvez le constater en visualisant les images, les classes sont regroupées à travers leur numéro (par exemple de 200 à 299, ce sont les images "Monuments").

- *Importer le tableau de mesure et créer un vecteur de label indiquant la classe sous forme d'un chiffre de chaque image : la classe de l'image de nom "i.jpg" est contenue dans label[i].*

1.1.2 Mise en place d'un système de discrimination basée structure Full-Connected

Nous proposons de mettre en place un système de discrimination qui doit déterminer à partir d'un ensemble d'apprentissage la catégorie de l'image inconnue. Vous allez extraire aléatoirement quelques images de la base de données ainsi que les mesures associées. Ces images tests vont constituer vos images "inconnues", le reste du tableau constitue l'ensemble d'apprentissage.

La structure de discrimination à mettre en place sera de type Perceptron Multi-couches. Pour cela, vous utilisez le framework Keras/TensorFlow, les hyperparamètres sont laissés libres.

- Mettre en place un système de discrimination qui pour la présentation d'une image inconnue et de son vecteur de mesures associé propose une classe (Jungle, Plage, Monuments, Bus, Dinosaures, Eléphants, Fleurs, Chevaux, Montagne et Plats).
- Tester la procédure de classification avec les mesures et différentes images inconnues. Analyser les résultats (matrice de confusion, taux d'erreur).
- Comparer avec différents hyperparamètres.

1.1.3 Approche "Deep" (basée Data)

Nous proposons de mettre en place une stratégie entièrement basée sur l'apprentissage, à savoir que les descripteurs vont être construits pas des couches de convolution.

- *Mettre en place un système de classification qui a pour données d'entrée une image et qui calcule les descripteurs par des couches de convolution.*

Par exemple :

```
model = Sequential()  
model.add(Conv2D(32, (3, 3), input_shape=(256,256, 3)))  
model.add(Activation('relu'))  
model.add(MaxPooling2D(pool_size=(2, 2)))
```

Nous appliquons ensuite une structure “Full Connected “ pour prendre la décision donc pour la présentation d'une image inconnue propose une classe (Jungle, Plage, Monuments, Bus, Dinosaur, Eléphants, Fleurs, Chevaux, Montagne et Plats).

```
model.add(Flatten())  
model.add(Dense(10, activation='softmax'))
```

- *Tester des structures simples. Etudier l'influence des paramètres, l'évolution de la fonction de coût.*
- *Comparer avec les résultats avec les méthodes basées caractéristiques.*